

CAPÍTULO 4

Clases y Objetos

Básicos

La gran idea de la programación orientada a objetos: modelar el mundo con clases, dar vida a objetos y proteger su estado.



En este capítulo aprenderás a...

- ✓ Distinguir entre una clase (el plano) y un objeto (la construcción real).
- ✓ Definir atributos (estado) y métodos (comportamiento) y darles uso.
- ✓ Crear objetos con constructores, sobrecargarlos y encadenarlos con `this(...)`.
- ✓ Usar la palabra `this`, los miembros `static` y los bloques de inicialización.
- ✓ Aplicar encapsulación con `private`, `getters/setters` y validaciones.
- ✓ Elegir el modificador de acceso correcto: `public`, `private`, `protected` y el de paquete.



Cómo está organizado este capítulo

Cada tema sigue el mismo ritmo: una explicación clara con una analogía, ejemplos de código comentados, una o dos cajas de aviso, y un “Reto rápido” para que practiques de inmediato. Al final encontrarás un proyecto integrador y una tanda de retos mayores. No avances sin resolver los retos rápidos: son el andamio que sostiene lo siguiente.

0 Antes de empezar: del mundo real al código

Mira a tu alrededor. Todo lo que ves son “objetos”: una silla, un teléfono, un coche. Cada uno tiene características (un color, un tamaño, un estado) y puede hacer cosas (sonar, encenderse, moverse). La Programación Orientada a Objetos (POO) es, sencillamente, una forma de programar que imita esa manera natural de ver el mundo.

En lugar de tener datos sueltos por un lado y funciones por otro, la POO los agrupa en objetos: paquetes que reúnen lo que algo ES (sus datos) con lo que algo HACE (sus acciones). Esto hace que los programas grandes sean más fáciles de entender, mantener y ampliar.



Las cuatro grandes ideas de la POO

Abstracción: quedarse con lo esencial e ignorar el resto.

Encapsulación: proteger los datos y exponer solo lo necesario. (la veremos en este capítulo)

Herencia: crear clases nuevas a partir de otras. (próximo capítulo)

Polimorfismo: que un mismo mensaje se comporte distinto según el objeto. (próximo capítulo)

Este capítulo es la base de todo: aprenderás a crear tus propias clases y objetos, el ladrillo con el que se construye absolutamente todo en Java.

1 Clases y objetos: el plano y la construcción

Esta es la distinción más importante de todo el capítulo, así que vamos despacio. Imagina los planos de un arquitecto para una casa. Los planos NO son una casa: son las instrucciones que describen cómo será. Con un único plano puedes construir muchas casas, todas con la misma estructura pero pintadas de colores distintos o amuebladas a su manera.

La clase es el plano; los objetos, las construcciones

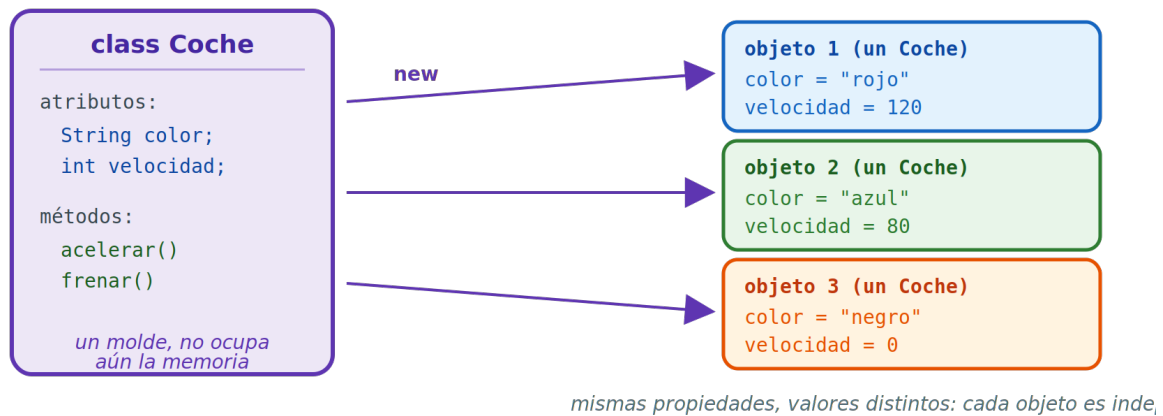


Figura 4.1 — Una clase (el plano) sirve para crear muchos objetos (las construcciones).

1.1 La clase: el plano

Una clase es la plantilla o el molde. Describe qué características (atributos) y qué acciones (métodos) tendrán los objetos creados a partir de ella, pero por sí sola no “existe” en memoria como algo concreto: es solo la definición.

1.2 El objeto: la construcción real

Un objeto es una instancia concreta de una clase: una “casa” construida a partir del plano. Ocupa memoria, tiene valores reales en sus atributos y puede ejecutar sus métodos. De una misma clase puedes crear todos los objetos que quieras, y cada uno es independiente de los demás.



La frase para recordarlo

La clase se escribe una vez; los objetos se crean muchas veces. Clase = molde. Objeto = lo que sale del molde.

1.3 Tu primera clase

Definamos una clase Coche. Por ahora solo tendrá dos atributos. Fíjate en la convención: el nombre de la clase empieza en mayúscula (PascalCase).

```
Coche.java

1 public class Coche {
2     // Atributos: Las características de cada coche
3     String color;
```

```
4     int velocidad;
5 }
```

1.4 Crear objetos con new

Para construir un objeto a partir de la clase usamos la palabra clave `new`. El resultado lo guardamos en una variable que actúa como un “mando a distancia” para controlar ese objeto.

```
Principal.java

1 public class Principal {
2     public static void main(String[] args) {
3         Coche miCoche = new Coche(); // creamos un objeto Coche
4         Coche otro    = new Coche(); // otro objeto, independiente
5
6         System.out.println(miCoche.color); // null (aún sin valor)
7     }
8 }
```

Un objeto = estado + comportamiento

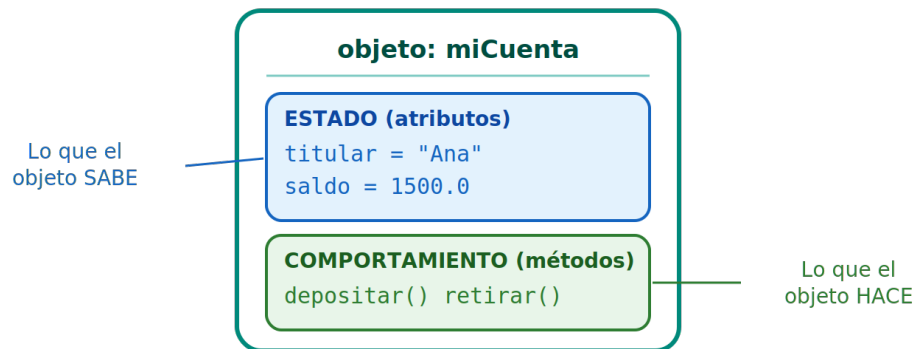


Figura 4.2 — Cada objeto reúne su estado (atributos) y su comportamiento (métodos).

🎯 Reto rápido

Crea una clase `Mascota` con dos atributos: `nombre (String)` y `edad (int)`. Luego, en un `main`, crea dos objetos `Mascota` distintos.

Pista: `class Mascota { String nombre; int edad; } y luego new Mascota();`

2 Atributos: el estado del objeto

Los atributos (también llamados campos o variables de instancia) son las características que describen a un objeto. Son lo que el objeto “sabe” de sí mismo: su estado. Cada objeto tiene su propia copia de los atributos, por lo que dos objetos de la misma clase pueden tener valores totalmente distintos.

2.1 Declarar atributos

Se declaran dentro de la clase, pero fuera de cualquier método, indicando su tipo y su nombre.

```
Estudiante.java

1 public class Estudiante {
2     String nombre;    // atributo de tipo texto
3     int edad;         // atributo de tipo entero
4     double promedio;  // atributo de tipo decimal
5     boolean activo;   // atributo de tipo booleano
6 }
```

2.2 Acceder y modificar atributos: el operador punto

Desde fuera de la clase, accedemos a los atributos de un objeto con el operador punto (.): objeto.atributo. Sirve tanto para leer su valor como para asignarle uno nuevo.

```
UsoEstudiante.java

1 Estudiante ana = new Estudiante();
2
3 // Asignar valores (escribir):
4 ana.nombre = "Ana";
5 ana.edad = 20;
6 ana.promedio = 8.7;
7
8 // Leer valores:
9 System.out.println(ana.nombre + " tiene " + ana.edad + " años");
```

Consola / Terminal
Ana tiene 20 años

2.3 Valores por defecto

Si creas un objeto y no asignas valores, sus atributos NO quedan vacíos ni con basura: Java les da un valor por defecto según su tipo.

Tipo de atributo	Valor por defecto
números (int, double, long...)	0 / 0.0
boolean	false
char	'\u0000' (carácter nulo)
objetos (String y otros)	null

⚠ Cuidado con null

Un atributo de tipo objeto sin inicializar vale null (“nada”). Si intentas usar un método sobre él (por ejemplo, `ana.nombre.length()` cuando `nombre` es null), obtendrás el famoso `NullPointerException`. Más adelante, los constructores nos ayudarán a evitar este problema dando valores iniciales.

🎯 Reto rápido

Añade a tu clase `Mascota` un tercer atributo boolean `vacunada`. Crea un objeto, imprime su valor SIN asignarlo (debería salir `false`) y luego cámbialo a `true` e imprímelo de nuevo.

Pista: *boolean por defecto es false; usa `mascota.vacunada = true;`*

3 Métodos: el comportamiento del objeto

Si los atributos son lo que el objeto “sabe”, los métodos son lo que el objeto “puede hacer”: su comportamiento. Un método de instancia es una función que vive dentro de la clase y que normalmente trabaja con los atributos del objeto que lo invoca.

3.1 Definir métodos de instancia

Los métodos van dentro de la clase. Un método puede leer y modificar los atributos del propio objeto directamente, por su nombre.

```
Coche.java

1 public class Coche {
2     String color;
3     int velocidad;
4
5     // Método: sube La velocidad
6     void acelerar(int cantidad) {
7         velocidad = velocidad + cantidad;
8     }
9
10    // Método: baja La velocidad sin pasar de 0
11    void frenar(int cantidad) {
12        velocidad = velocidad - cantidad;
13        if (velocidad < 0) {
14            velocidad = 0;
15        }
16    }
17
18    // Método que devuelve información
19    String describir() {
20        return "Coche " + color + " a " + velocidad + " km/h";
21    }
22 }
```

3.2 Llamar a los métodos de un objeto

Igual que con los atributos, usamos el operador punto: objeto.metodo(argumentos). El método actúa sobre ESE objeto en concreto.

```
UsoCoche.java

1 Coche miCoche = new Coche();
2 miCoche.color = "rojo";
3
4 miCoche.acelerar(50);           // velocidad pasa a 50
5 miCoche.acelerar(30);           // velocidad pasa a 80
6 miCoche.frenar(100);            // baja, pero se queda en 0 (no negativo)
7
8 System.out.println(miCoche.describir());

Consola / Terminal
Coche rojo a 0 km/h
```



Cada objeto actúa sobre SU propio estado

Si tienes dos coches y aceleras uno, el otro no cambia. Cuando llamas a `miCoche.acelerar(50)`, el método modifica la velocidad de `miCoche`, no la de otro coche. Cada objeto es su propio mundo.



Reto rápido

Añade a tu clase `Mascota` un método `cumplirAnios()` que sume 1 a la edad, y un método `String saludar()` que devuelva "Hola, soy " + nombre. Pruébalos desde un `main`.

Pista: `void cumplirAnios() { edad = edad + 1; }` y `String saludar() { return "Hola, soy " + nombre; }`

4 Constructores: nacer ya configurado

Hasta ahora creábamos un objeto vacío y luego le asignábamos valores uno por uno. Es tedioso y fácil de olvidar. Un constructor resuelve esto: es un método especial que se ejecuta automáticamente al crear el objeto con `new`, y sirve para dejarlo configurado desde el primer instante.

¿Qué pasa cuando escribes `new` ?



```
Cuenta c = new Cuenta ("Ana", 1500 );
```

Figura 4.3 — `new` reserva memoria, ejecuta el constructor y devuelve la referencia.



Reglas de un constructor

Se llama **EXACTAMENTE** igual que la clase.

No tiene tipo de retorno (ni siquiera `void`).

Se ejecuta solo, una vez, al hacer `new`.

4.1 El constructor por defecto

Si no escribes ningún constructor, Java te regala uno invisible y vacío (el constructor por defecto), que es justo lo que permitía que `new Coche()` funcionara antes. En cuanto escribes tu propio constructor, ese regalo desaparece.

4.2 Constructores con parámetros

Lo habitual es escribir un constructor que reciba los datos iniciales como parámetros y los guarde en los atributos:

```
Coche.java

1 public class Coche {
2     String color;
3     int velocidad;
4
5     // Constructor: mismo nombre que la clase, sin tipo de retorno
6     Coche(String colorInicial) {
7         color = colorInicial;
8         velocidad = 0;           // todo coche nace detenido
9     }
10 }
```

Ahora crear un coche es directo y seguro: nace con su color y velocidad ya definidos.

```

1 Coche rojo = new Coche("rojo");    // el constructor recibe "rojo"
2 System.out.println(rojo.color);    // rojo
3 System.out.println(rojo.velocidad); // 0

```

4.3 Sobrecarga de constructores

Igual que los métodos, los constructores se pueden sobrecargar: tener varios con distintos parámetros, para ofrecer distintas formas de crear el objeto. Java elige el adecuado según los argumentos que pases.

```

1 public class Coche {
2     String color;
3     int velocidad;
4
5     Coche() {                // sin datos: valores genéricos
6         color = "blanco";
7         velocidad = 0;
8     }
9
10    Coche(String color) {     // con color
11        this.color = color;
12        this.velocidad = 0;
13    }
14
15    Coche(String color, int velocidad) { // con color y velocidad
16        this.color = color;
17        this.velocidad = velocidad;
18    }
19 }

```

4.4 this(...): un constructor que llama a otro

Para no repetir código entre constructores sobrecargados, uno puede llamar a otro con `this(...)`. Debe ser la PRIMERA línea del constructor.

```

1 Coche() {
2     this("blanco", 0);    // reutiliza el constructor de abajo
3 }
4
5 Coche(String color, int velocidad) {
6     this.color = color;
7     this.velocidad = velocidad;
8 }

```



Buena práctica 2026

Centraliza la inicialización en el constructor “más completo” y haz que los demás lo llamen con `this(...)`. Así, si cambias la lógica de inicio, solo la tocas en un sitio. Es el patrón de “constructor telescópico” bien hecho.



Reto rápido

Dale a tu clase `Mascota` un constructor que reciba nombre y edad, y otro sin parámetros que use `this(...)` para crear una mascota llamada “Sin nombre” de edad 0.

Pista: `Mascota()` { `this("Sin nombre", 0);` } y `Mascota(String nombre, int edad)` { `this.nombre = nombre;`
`this.edad = edad;` }

5 La palabra clave this

Ya la viste aparecer en los constructores. `this` es una referencia al objeto actual, es decir, al objeto sobre el que se está ejecutando el método o constructor en este momento. Tiene dos usos principales.

5.1 Desambiguar atributo vs parámetro

Es muy común que un parámetro tenga el mismo nombre que un atributo (por claridad). En ese caso, dentro del método, el nombre “a secas” se refiere al parámetro, y `this.nombre` se refiere al atributo. Así evitas confusiones:

```
Persona.java

1 public class Persona {
2     String nombre;
3
4     Persona(String nombre) {
5         // "nombre" es el parámetro; "this.nombre" es el atributo
6         this.nombre = nombre;
7     }
8 }
```

⚠ Sin this, el clásico error silencioso

Si dentro del constructor escribes `nombre = nombre;` estás asignando el parámetro a sí mismo, ¡y el atributo se queda en null! El compilador no se queja, pero el objeto no se inicializa. Por eso `this` es imprescindible cuando los nombres coinciden.

5.2 this como referencia al objeto actual

`this` también sirve para pasar el objeto completo a otro método, o para encadenar llamadas devolviéndolo. Y, como viste, `this(...)` llama a otro constructor de la misma clase.

```
Cuenta.java

1 public class Cuenta {
2     double saldo;
3
4     // Devolver this permite encadenar llamadas (estilo "fluido")
5     Cuenta depositar(double monto) {
6         this.saldo += monto;
7         return this;           // devuelve el propio objeto
8     }
9 }
10
11 // Gracias a 'return this', se pueden encadenar:
12 // cuenta.depositar(100).depositar(50).depositar(25);
```

🎯 Reto rápido

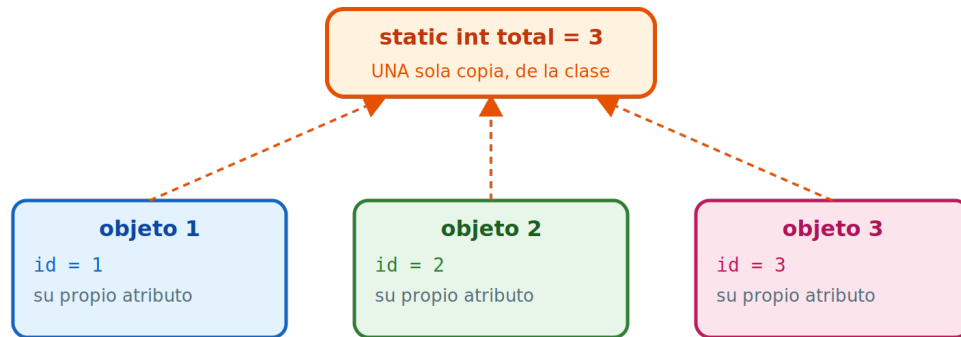
Reescribe el constructor de tu clase Mascota para que los parámetros se llamen igual que los atributos (nombre, edad) y usa this para asignarlos correctamente.

Pista: `this.nombre = nombre; this.edad = edad;`

6 Miembros estáticos (static)

Hasta ahora, cada objeto tenía su propia copia de cada atributo. Pero a veces queremos algo que pertenezca a la CLASE entera y sea compartido por todos los objetos. Para eso existe la palabra `static`.

static: lo que comparten TODOS los objetos



id es de cada objeto (instancia) · total pertenece a la clase y lo ven todos

Figura 4.4 — Un atributo `static` tiene una sola copia; los de instancia, una por objeto.

6.1 Atributos estáticos: compartidos

Un atributo `static` existe una sola vez, a nivel de clase, sin importar cuántos objetos crees. Es perfecto, por ejemplo, para llevar la cuenta de cuántos objetos se han creado.

```
● ● ● Coche.java

1 public class Coche {
2     static int totalCoches = 0; // COMPARTIDO por todos Los coches
3     String color;              // PROPIO de cada coche
4
5     Coche(String color) {
6         this.color = color;
7         totalCoches++;          // cada nuevo coche suma al total
8     }
9 }
```

```
● ● ● Conteo.java

1 new Coche("rojo");
2 new Coche("azul");
3 new Coche("verde");
4
5 // Se accede con el nombre de la CLASE, no de un objeto:
6 System.out.println(Coche.totalCoches); // 3
```

6.2 Métodos estáticos

Un método `static` pertenece a la clase y se llama sin necesidad de crear un objeto. Por eso NO puede usar atributos de instancia ni `this` (no hay ningún objeto concreto detrás). El método `main` es estático, y también lo son utilidades como `Math.sqrt()` o `Math.max()`.

```

Utilidades.java

1 public class Utilidades {
2     static int cuadrado(int n) {
3         return n * n;
4     }
5 }
6
7 // Se llama directamente con la clase, sin crear objetos:
8 int r = Utilidades.cuadrado(5); // 25

```

⚠ Lo que un método static NO puede hacer

Un método estático no puede acceder a atributos de instancia ni usar `this`, porque se ejecuta sin un objeto concreto. Si lo intentas, no compila. Regla práctica: usa `static` para utilidades que no dependen del estado de ningún objeto.

6.3 Constantes con `static final`

Combinando `static` (compartido) con `final` (no cambia) se crean constantes: valores fijos que pertenecen a la clase. Por convención se escriben en MAYÚSCULAS con guion bajo.

```

Circulo.java

1 public class Circulo {
2     static final double PI = 3.14159; // constante de clase
3     double radio;
4
5     double area() {
6         return PI * radio * radio;
7     }
8 }

```

🎯 Reto rápido

Añade a tu clase `Mascota` un atributo `static int totalMascotas` que aumente en el constructor, y compruébalo creando 3 mascotas e imprimiendo `Mascota.totalMascotas`.

Pista: `static int totalMascotas = 0;` y dentro del constructor: `totalMascotas++;`

7 Bloques estáticos e inicializadores

A veces la inicialización necesita más que una simple asignación. Java ofrece dos tipos de bloques especiales que se ejecutan automáticamente en momentos concretos.

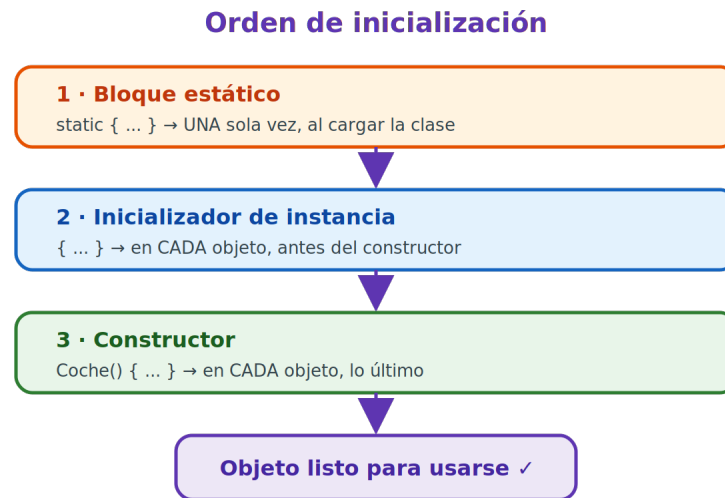


Figura 4.5 — El orden en que Java inicializa: estático, de instancia y constructor.

7.1 Bloque estático: `static { ... }`

Se ejecuta UNA sola vez, cuando la clase se carga por primera vez (antes de crear cualquier objeto). Sirve para preparar recursos compartidos o inicializar atributos estáticos complejos.

```
Config.java

1 public class Config {
2     static String entorno;
3
4     static {
5         // se ejecuta una vez, al cargar la clase
6         entorno = "producción";
7         System.out.println("Configuración cargada");
8     }
9 }
```

7.2 Inicializador de instancia: `{ ... }`

Un bloque sin `static`, dentro de la clase, se ejecuta en CADA objeto, justo antes del constructor. Es útil cuando varios constructores comparten una misma preparación inicial.

```
Jugador.java

1 public class Jugador {
2     int vidas;
3
4     {
5         vidas = 3; // se ejecuta en cada objeto, antes del constructor
6         System.out.println("Nuevo jugador creado");
7     }
8 }
```

```

7      }
8
9      Jugador() { }
10   }

```

7.3 El orden completo, demostrado

Cuando todo se combina, Java sigue siempre el mismo orden: primero el bloque estático (una vez), luego el inicializador de instancia y, por último, el constructor (estos dos, en cada objeto). Este ejemplo lo deja clarísimo:

```

Orden.java

1  public class Orden {
2      static { System.out.println("1. Bloque estático"); }
3      { System.out.println("2. Inicializador de instancia"); }
4      Orden() { System.out.println("3. Constructor"); }
5
6      public static void main(String[] args) {
7          System.out.println("--- creando primer objeto ---");
8          new Orden();
9          System.out.println("--- creando segundo objeto ---");
10         new Orden();
11     }
12 }

```

Consola / Terminal

```

1. Bloque estático
--- creando primer objeto ---
2. Inicializador de instancia
3. Constructor
--- creando segundo objeto ---
2. Inicializador de instancia
3. Constructor

```



Observa el detalle

El bloque estático aparece UNA sola vez, aunque creamos dos objetos. En cambio, el inicializador de instancia y el constructor se repiten por cada objeto. Esa es exactamente la diferencia entre “de clase” y “de instancia”.

🎯 Reto rápido

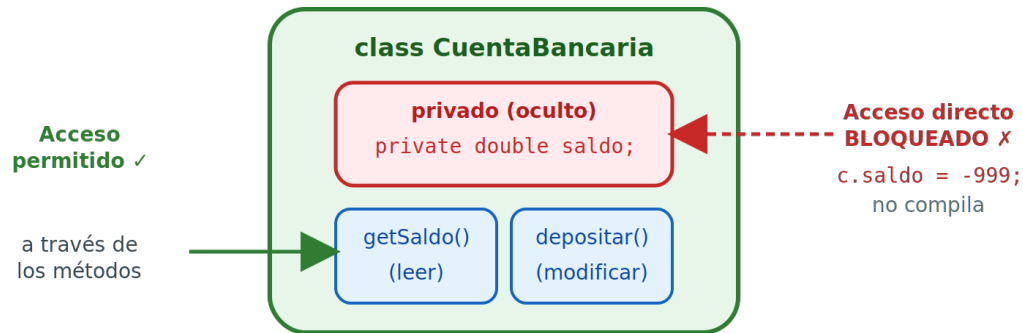
En tu clase Mascota, añade un bloque estático que imprima “Clínica veterinaria abierta” y un inicializador de instancia que imprima “Nueva ficha”. Crea dos mascotas y observa el orden y la cantidad de veces que aparece cada mensaje.

Pista: El estático sale una vez; “Nueva ficha”, una por cada objeto.

8 Encapsulación: proteger el estado

Hasta ahora, cualquiera podía tocar nuestros atributos directamente: `coche.velocidad = -500`; sería perfectamente válido, aunque no tenga sentido. La encapsulación es el principio de ocultar los datos internos de un objeto y permitir acceder a ellos solo a través de métodos controlados. Así el objeto protege su propia coherencia.

Encapsulación: el estado, protegido



Los datos se tocan SOLO por métodos controlados, que pueden validar antes de cambiar.

Figura 4.6 — Los datos privados se modifican solo por métodos públicos que pueden validar.

8.1 ¿Por qué encapsular?

Piensa en un cajero automático. No puedes meter la mano y cambiar tu saldo directamente; debes usar las operaciones que el banco te ofrece (depositar, retirar), y esas operaciones comprueban que todo sea válido. Encapsular es exactamente eso: cerrar el acceso directo y obligar a pasar por la “puerta” oficial.

- Protege los datos de valores inválidos (una edad negativa, un saldo imposible).
- Permite cambiar la implementación interna sin romper el código que usa la clase.
- Hace la clase más fácil de entender: queda claro qué se puede hacer con ella.

8.2 El patrón: `private` + getters/setters

La receta de la encapsulación tiene dos pasos: (1) marcar los atributos como `private` para que nadie los toque desde fuera; (2) ofrecer métodos públicos para leerlos (getters, `get...`) y modificarlos (setters, `set...`).

```
Persona.java

1 public class Persona {
2     private String nombre; // privado: nadie lo toca directo
3     private int edad;
4
5     // GETTER: permite leer el atributo
6     public String getNombre() {
7         return nombre;
8     }
9
10    // SETTER: permite modificar el atributo
11    public void setNombre(String nombre) {
```

```

12     this.nombre = nombre;
13 }
14
15 public int getEdad() {
16     return edad;
17 }
18 }

```

Desde fuera, ya no se accede al atributo, sino a sus métodos:

```

• • •  UsoPersona.java

1  Persona p = new Persona();
2  p.setNombre("Ana");           // en vez de p.nombre = ...
3  System.out.println(p.getNombre()); // en vez de p.nombre
4
5  // p.nombre = "X"; // ❌ ERROR: nombre es private, no compila

```

8.3 La verdadera ventaja: validar en el setter

Aquí brilla la encapsulación. Como toda modificación pasa por el setter, podemos poner reglas que impidan estados imposibles. El objeto se vuelve “inteligente” y rechaza datos absurdos:

```

• • •  Persona.java

1  public void setEdad(int edad) {
2      if (edad >= 0 && edad <= 120) {
3          this.edad = edad;           // solo acepta valores razonables
4      } else {
5          System.out.println("Edad inválida, se ignora: " + edad);
6      }
7  }

```

```

▶ Consola / Terminal

// p.setEdad(25);    -> ok, edad = 25
// p.setEdad(-7);    -> Edad inválida, se ignora: -7

```



Buena práctica 2026

No crees getters y setters “porque sí” para todos los atributos. Expón solo lo que el resto del programa necesita de verdad, y aprovecha los setters para validar. Si un dato no debe cambiar tras crearse, márcalo final y no le pongas setter. Menos puertas abiertas = menos errores.



Reto rápido

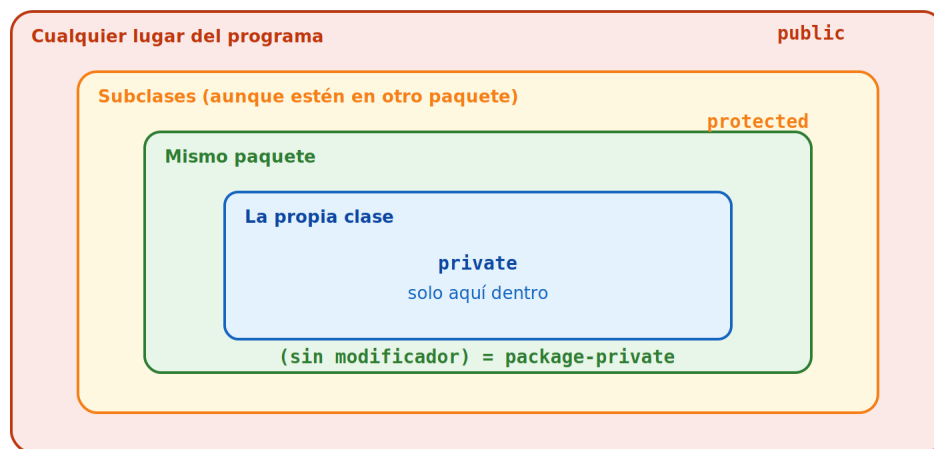
Convierte tu clase Mascota en encapsulada: pon nombre y edad como private y añade sus getters y setters. En setEdad, rechaza edades negativas.

Pista: `private int edad; + public void setEdad(int edad){ if(edad>=0) this.edad=edad; }`

9 Modificadores de acceso

Los modificadores de acceso son las palabras que controlan QUIÉN puede ver y usar cada atributo, método o clase. Son la herramienta concreta con la que aplicamos la encapsulación. Java tiene cuatro niveles, de más cerrado a más abierto.

¿Hasta dónde llega cada modificador?



De más restrictivo (*private*, centro) a más abierto (*public*, todo el programa).

Figura 4.7 — Cada modificador define hasta dónde llega la visibilidad.

9.1 Los cuatro niveles

Modificador	¿Quién puede acceder?
<code>private</code>	Solo desde la propia clase. El más cerrado.
<code>(sin modificador)</code>	Desde el mismo paquete. Se llama “package-private” o “de paquete”.
<code>protected</code>	Desde el mismo paquete y desde las subclases (lo verás con herencia).
<code>public</code>	Desde cualquier parte del programa. El más abierto.

9.2 Tabla de visibilidad

Esta tabla resume hasta dónde llega cada nivel. Un ✓ significa “puede acceder”.

Acceso desde...	<code>private</code>	<code>(paquete)</code>	<code>protected</code>	<code>public</code>
La misma clase	✓	✓	✓	✓
Otra clase del mismo paquete	—	✓	✓	✓
Una subclase en otro paquete	—	—	✓	✓
Cualquier otra clase	—	—	—	✓

9.3 La regla práctica



Buena práctica 2026

Empieza siempre por lo más restrictivo posible y abre solo cuando haga falta.

Atributos → private casi siempre.

Métodos que forman parte de cómo se usa la clase → public.

Detalles internos de ayuda → private.

Reserva protected para cuando trabajes con herencia. El de paquete es poco común en el día a día.



Reto rápido

Revisa tu clase Mascota: ¿están sus atributos en private? ¿Sus getters/setters y métodos útiles en public? Ajusta los modificadores para que solo se acceda a los datos por métodos.

Pista: *Atributos private; getters, setters y métodos de uso public.*

★ Proyecto integrador: CuentaBancaria

Reunamos TODO lo aprendido en una sola clase realista. Una CuentaBancaria que: encapsula su saldo (private), se crea con un constructor, valida las operaciones, lleva un contador estático de cuentas y usa una constante. Léela con calma: contiene casi todos los conceptos del capítulo.

```
CuentaBancaria.java

1 public class CuentaBancaria {
2     // --- Atributos de instancia (privados: encapsulación) ---
3     private String titular;
4     private double saldo;
5
6     // --- Miembros estáticos (compartidos por todas las cuentas) ---
7     private static int totalCuentas = 0;
8     public static final double SALDO_MINIMO = 0.0;    // constante
9
10    // --- Constructor ---
11    public CuentaBancaria(String titular, double saldoInicial) {
12        this.titular = titular;
13        this.saldo = (saldoInicial >= SALDO_MINIMO) ? saldoInicial : 0;
14        totalCuentas++;    // una cuenta más en el banco
15    }
16
17    // --- Métodos de comportamiento (con validación) ---
18    public void depositar(double monto) {
19        if (monto > 0) {
20            saldo += monto;
21        } else {
22            System.out.println("El depósito debe ser positivo.");
23        }
24    }
25
26    public void retirar(double monto) {
27        if (monto > 0 && monto <= saldo) {
28            saldo -= monto;
29        } else {
30            System.out.println("Retiro inválido o saldo insuficiente.");
31        }
32    }
33
34    // --- Getters (lectura controlada) ---
35    public double getSaldo() { return saldo; }
36    public String getTitular() { return titular; }
37    public static int getTotalCuentas() { return totalCuentas; }
38 }
```

Y así se usa desde un programa principal:

```
Banco.java

1 public class Banco {
2     public static void main(String[] args) {
3         CuentaBancaria c1 = new CuentaBancaria("Ana", 1000);
```

```

4      CuentaBancaria c2 = new CuentaBancaria("Luis", 500);
5
6      c1.depositar(250);
7      c1.retirar(2000);           // será rechazado
8      c1.retirar(300);
9
10     System.out.println(c1.getTitular() + ": " + c1.getSaldo());
11     System.out.println("Cuentas abiertas: "
12                        + CuentaBancaria.getTotalCuentas());
13 }
14 }

```

▶ Consola / Terminal

```

Retiro inválido o saldo insuficiente.
Ana: 950.0
Cuentas abiertas: 2

```



Lo que demuestra este proyecto

- ✓ titular y saldo son private: nadie los corrompe desde fuera (encapsulación).
- ✓ El constructor deja la cuenta lista y válida desde el inicio.
- ✓ depositar y retirar validan: el objeto protege su propia coherencia.
- ✓ totalCuentas es static: lo comparten todas las cuentas del banco.
- ✓ SALDO_MINIMO es static final: una constante de la clase.

🚩 Retos del capítulo

Estos retos integran varios conceptos a la vez. Diseña primero los atributos y métodos en papel, luego escribe la clase y, por último, una clase con main para probarla.

Reto 1 · Clase Rectangulo [básico]

Crea una clase Rectangulo con atributos privados base y altura, un constructor, getters/setters y dos métodos: area() y perimetro().

💡 **Pista:** $area = base * altura$; $perimetro = 2 * (base + altura)$.

Reto 2 · Clase Libro [básico]

Modela un Libro con titulo, autor y un boolean prestado. Añade métodos prestar() y devolver() que cambien el estado, e impide prestar un libro ya prestado.

💡 **Pista:** En prestar(), comprueba if (!prestado) antes de cambiarlo a true.

Reto 3 · Contador de objetos [medio]

Crea una clase Usuario que asigne automáticamente un id único a cada objeto (1, 2, 3...) usando un atributo static, y un método que devuelva cuántos usuarios se han creado.

💡 **Pista:** Un static int siguienteId y un atributo de instancia id = siguienteId++.

Reto 4 · Termómetro encapsulado [medio]

Diseña una clase Termometro con un atributo privado celsius y un setter que rechace temperaturas por debajo de -273.15. Añade un getter que devuelva la temperatura en Fahrenheit.

💡 **Pista:** $F = C * 9/5 + 32$. Valida en el setter antes de asignar.

Reto 5 · Constructores telescópicos [medio]

Crea una clase Pizza con tamaño (String) e ingredientes (int). Ofrece tres constructores: completo, solo con tamaño (ingredientes = 0) y vacío ("mediana", 0). Encadénalos con this(...).

💡 **Pista:** El vacío llama al de tamaño, y este al completo, todos con this(...).

Reto 6 · Mini-sistema de inventario (proyecto) [avanzado]

Crea una clase Producto (nombre, precio, stock, todos privados) con validación en los setters (precio y stock nunca negativos), métodos vender(cantidad) y reponer(cantidad), un contador static de productos y una constante con el IVA. Pruébala con un main que simule varias ventas, incluida una venta sin stock suficiente.

💡 **Pista:** Combina encapsulación, static, final, validación y this. Reutiliza la lógica de CuentaBancaria del proyecto integrador.

✓ Resumen del capítulo

Lo esencial que te llevas

- ✓ Una clase es el plano; un objeto es una instancia concreta creada con `new`.
- ✓ Los atributos guardan el estado del objeto; los métodos definen su comportamiento.
- ✓ El constructor inicializa el objeto al crearlo; se puede sobrecargar y encadenar con `this(...)`.
- ✓ `this` es la referencia al objeto actual; resuelve la ambigüedad atributo/parámetro.
- ✓ `static` crea miembros de clase compartidos; `static final` crea constantes.
- ✓ El orden de inicialización es: bloque estático (una vez), inicializador de instancia y constructor.
- ✓ La encapsulación protege el estado con `private` + `getters/setters`, validando en los `setters`.
- ✓ Los modificadores `private`, de paquete, `protected` y `public` controlan la visibilidad, de menos a más.

En el próximo capítulo: *herencia, polimorfismo, clases abstractas e interfaces.*